HISTORY OF NLP

As we all know Natural Language refers to the language spoken by people e.g. English, Hindi, Marathi as opposed to artificial/programming languages, like C, C++, Java, etc.

Natural Language Processing (NLP) is a field of research which determines the way computers can be used to understand and manage natural language text or speech to do useful things.

Natural Language Processing (NLP) a subset technique of Artificial Intelligence which is used to narrow the communication gap between the Computer and Human. It is originated from the idea of Machine Translation (MT) which came to existence during the 1950. The primary aim was to translate one human language to another human language, for example, Russian language to English language using the brain of the Computers but after that, the thought of conversion of human language to computer language and vice-versa emerged, so that communication with this input into a useful form of representation. The field of NLP is primarily concerned with getting computers to perform interesting and useful tasks with human languages. The field of NLP is secondarily concerned with helping us come to a better understanding of human language.

As stated above the idea had emerged from the need for Machine Translation in the 1950s. Then the original language was English and Russian. But the use of other words such as Chinese also came into existence in the initial period of the 1960s.

In the 1960s, the NLP got a new life when the idea and need of Artificial Intelligence emerged.

In 1978 LUNAR is developed by W.A woods; it could analyze, compare and evaluate the chemical data on a lunar rock and soil composition that was accumulating as a result of Apollo moon missions and can answer the related question.

In the 1980s the area of computational grammar became a very active field of research which was linked with the science of reasoning for meaning and considering the user‘s beliefs and intentions.

In the period of 1990s, the pace of growth of NLP increased. Grammars, tools and Practical resources related to NLP became available with the parsers. Probabilistic and data-driven models had become quite standard by then.

In 2000, Engineers had a large amount of spoken and textual data available for creating systems. Today, large amount of work is being done

in the field of NLP using Machine Learning or Deep Neural Networks in general, where we are able to create state-of-the-art models in text classification, Question and Answer generation, Sentiment Classification, etc.

1.0 GENERIC NLP SYSTEM

Natural language Processing should start with some input and ends with effective and accurate output. The possible inputs to an NLP are quite broad. Language can be in a variety of forms such as the paragraphs of text, commands that are typed directly to a computer system, etc. The input language might be given to the system a sentence at a time or it might be multiple sentences all at once. The inputs for natural language processor can be typed input, message text or speech. NLP systems have some kind of pre-processor. Data preprocessing involves preparing and cleaning text data for machines to be able to analyze it. Preprocessing puts data in workable form and highlights features in the text that an algorithm can work with. Preprocessing does dictionary lookup, morphological analysis, lexical substitutions, and part-of-speech assignment.

from a system that incorporates an NLP might be an answer from a database, a command to change some data in a database, a spoken response, Semantics, Part of speech, Morphology of word, Semantics of the word or some other action on the part of the system. Remember these are the output of the system as a whole, not the output of the NLP component of the system.

Figure 1.2.1 shows a generic NLP system and its input-output variety. Figure 1.2.2 shows a typical view of what might be inside the NLP box of Figure 1.2.1. Each of the boxes in Figure 1.2.2 represents one of the types of processing that make up an NLP analysis.

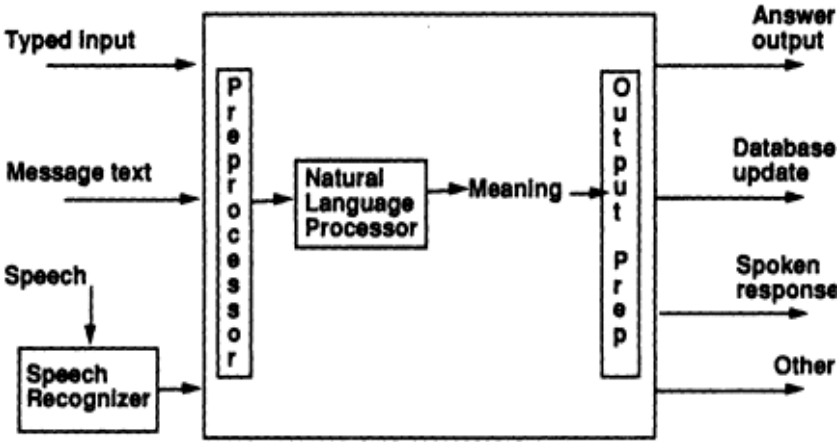


Figure 1.2.1 Generic NLP system.

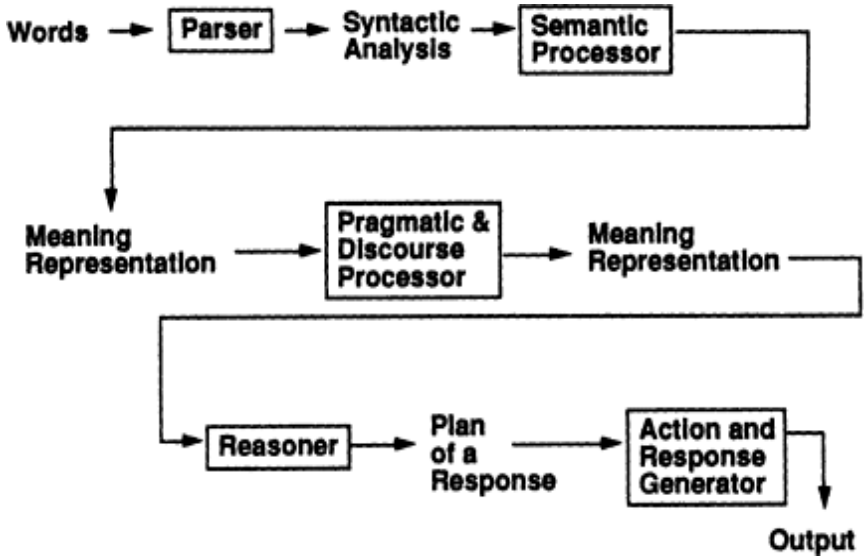


Figure 1.2.2 Pipeline view of the components of a generic NLP system.

There are two components of Natural Language Processing

a. Natural Language Understanding (NLU)

- NLU takes some spoken/typed sentence and working out what it means.
- Here different level of analysis required such as morphological analysis, syntactic analysis, semantic analysis, discourse analysis, …

b. Natural Language Generation (NLG)

- NLG takes some formal representation of what you want to say and working out a way to express it in a natural (human) language (e.g., English)
- Here different level of synthesis required: deep planning (what to say), syntactic generation

Difference between NLU and NLG

NLU	NLG
It is the process of reading and interpreting language.	It is the process of writing or generating language.

NLU explains the meaning behind the written text or speech in natural language	NLG generates the natural language using machines.
NLU draws facts from the natural language using various tools and technologies such as parsers, POS taggers, etc,	NLG uses the insights generated from parsers, POS tags, etc. to generate the natural language.
NLU understands the human language and converts it into data	NLG uses the structured data and generates meaningful narratives out of it

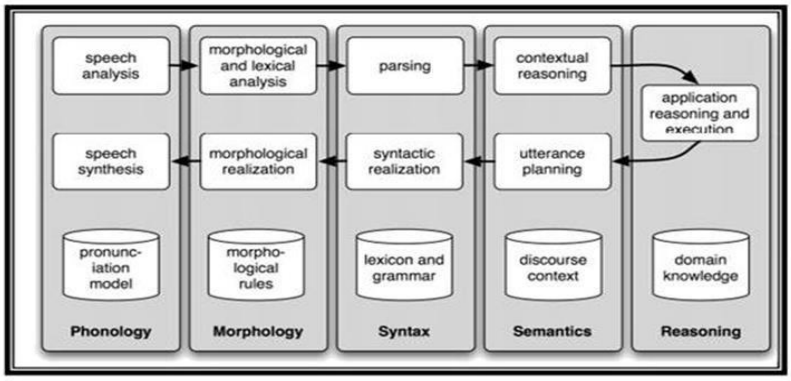


Figure 1.3.1 Levels of NLP

1. **Phonology:**
   It concerned with interpretation of speech sound within and across words.

2. **Morphology:**
   It deals with how words are constructed from more basic meaning units called morphemes. A morpheme is the primitive unit of meaning in a language. For example, "truth+ful+ness".

3. **Syntax:**
   It concerns how words can be put together to form correct sentences and determines what structural role each word plays in the sentence and what phrases are subparts of other phrases.

   For example, "the dog ate my homework"

4. **Semantics:**
   It is a study of the meaning of words and how these meaning combine in sentences to form sentence meaning. It is study of context- independent meaning. For example, plant: industrial plant/ living organism

   Pragmatics concerns with how sentences are used in different situations and how it affects the interpretation of the sentence. Discourse context deals with how the immediately preceding sentences affect the interpretation of the next sentence. For example, interpreting pronouns and interpreting the temporal aspects of the information.

5. **Reasoning:**
   To produce an answer to a question which is not explicitly stored in a database; Natural Language Interface to Database (NLIDB) carries out reasoning based on data stored in the database. For example, consider the database that holds the student academic information, and user posed a query such as: 'Which student is likely to fail in Science subject?' To answer the query, NLIDB needs a domain expert to narrow down the reasoning process.

KNOWLEDGE IN LANGUAGE PROCESSING

A natural language understanding system must have detailed information about what the words mean, how words combine to form sentences, how word meanings combine to form sentence meanings and so on.

What distinguishes these language processing applications from other data processing systems is their use of knowledge of language.

For example consider the Unix wc program, which is used to count the total number of bytes, words, and lines in a text file. When used to count lines and bytes, wc is an ordinary data processing application. However, when it is used to count the words in a file it requires information about what it means to be a word, and thus becomes a language processing system. Of course, wc is an extremely simple system with an extremely limited and impoverished knowledge of language.

The different forms of knowledge required for natural language understanding are given below:

Phonetic and Phonological Knowledge

Phonetics is the study of language at the level of sounds while phonology is the study of combination of sounds into organized units of speech, the formation of syllables and larger units. Phonetic and phonological knowledge are necessary for speech based systems as they are concerned with how words are related to the sounds that realize them.

Morphological Knowledge

Morphology concerns with word formation. It's a study of the patterns of formation of words by the combination of sounds into minimal distinctive units of meaning called mophemes. Morphological knowledge deals with how words are constructed from morphemes.

Syntactic Knowledge

Syntax is the level at which we study how words combine to form phrases, phrases combine to form clauses and clauses join to make sentences. Syntactic analysis concerns sentence formation. It's concerned with how words can be put together to form correct sentences. It also determines what structural role each word plays in the sentence and what phrases are subparts of what other phrases.

Semantic Knowledge

It deals with meanings of the words and sentences. This is the study of context independent meaning that is the meaning a sentence has, no matter in which context it is used. Defining the meaning of a sentence is very difficult due to the ambiguities involved.

Pragmatic Knowledge

Pragmatics is the extension of the meaning or semantics. Pragmatics concerned with the contextual aspects of meaning in particular situations. It concerned with how sentences are used in different situations and how use affects the interpretation of the sentence.

Discourse Knowledge

Discourse concerns connected sentences. It's a study of chunks of language which are bigger than a single sentence. Discourse language concerned with inter-sentential links means how the immediately preceding sentences affect the interpretation of the next sentence. Discourse knowledge is important for interpreting pronouns and temporal aspects of the information conveyed.

World Knowledge

Word knowledge is nothing but everyday knowledge that all speakers share about the world. It includes the general knowledge about the structure of the world and what each language user must know about the other user's beliefs and goals. This is essential to make the language understanding much better.

AMBIGUITY IN NATURAL LANGUAGE

Ambiguity can occur at all NLP levels. It is a property of linguistic expressions. If an expression (word/phrase/sentence) has more than one meaning we can refer it as ambiguous.

For eg: Consider the sentence,
"The chicken is ready to eat."
The meaning in the above phrase can be,
The chicken (food) is ready to be eaten or the chicken (bird) is ready to be feeder
Consider another sentence,
"There was not a single man at the party."
The meaning in this case can be Lack of men altogether or Lack of bachelors at the party.
There are different forms of ambiguity that are relevant in natural language processing.

1. **Lexical Ambiguity:** It's the ambiguity of a single word. A word can be ambiguous with respect to its syntactic class. Eg: book, study.

   For eg: The word "silver" can be used as, an adjective, a noun or a verb.

   - She made a silver speech.
   - She bagged two silver medals.
   - His worries had silvered his hair.

Lexical ambiguity can be resolved by Lexical category disambiguation ie, parts-of speech tagging. As many words may belong to more than one lexical category. Part-of speech tagging is the process of assigning a part-of-speech or lexical category such as a noun, pronoun, verb, preposition, adverb, adjective etc. to each word in a sentence.

Lexical Semantic Ambiguity:

The type of lexical ambiguity, which occurs when a single word is associated with multiple interpretations. Eg: fast, bat, bank, pen, cricket etc.

For eg: 1. The tank was full of water.

2. I saw a military tank.

Words have multiple meanings for such sentences. Consider the sentence "I saw a bat."

Possible meaning of the words which changes the contexts of the sentence are:

- bat flying mammal / wooden club?
- saw past tense of "see" /present tense of "saw " (to cut with a saw.)

The occurrence of tank in both sentences corresponds to the syntactic category noun, but their meanings are different. Lexical Semantic ambiguity resolved using word sense disambiguation (WSD) techniques, where WSD aims at automatically assigning the meaning of the word in the context in a computational manner.

2. **Syntactic Ambiguity:** A sentence can be parsed in different ways. For example, “He lifted the beetle with red cap” or “Did he use cap to lift the beetle”.

The structural ambiguities were syntactic ambiguities

Structural ambiguity is of two kinds: i) Scope Ambiguity ii) Attachment Ambiguity

– **Scope Ambiguity**: Scope ambiguity involves operators and quantifiers.

For example: Old men and women were taken to safe locations.

The scope of the adjective (i.e., the amount of text it qualifies) is ambiguous. That is, whether the structure (old men and women) or ((old men) and women)?

The scope of quantifiers is often not clear and creates ambiguity.

Every man loves a woman

The meaning can be, For every man there is a woman and also it can be there is one particular woman who is loved by every man.

- **Attachment Ambiguity**

A sentence has attachment ambiguity if a constituent fits more than one position in a parse tree. Attachment ambiguity arises from uncertainty of attaching a phrase or clause to a part of a sentence.

Consider the example:

The man saw the girl with the telescope.

It is ambiguous whether the man saw her through his telescope or he saw a girl carrying a telescope. The meaning is dependent on whether the preposition 'with' is attached to the girl or the man.

Consider the example:

Buy books for children and attach to the verb buy

Preposition Phrase 'for children' can be either adverbial or adjectival and attach to the object noun books.

3. **Semantic Ambiguity**: This type of ambiguity is typically related to the interpretation of sentence. Even after the syntax and the meanings of the individual words have been resolved, there are two ways of reading the sentence.

Consider example: "Seema loves mother and Sriya does too"

The interpretations can be Sriya loves Seema's mother Sriya likes her own mother.

Semantic ambiguities born from the fact that generally a computer is not in a position to distinguishing what is logical from what is not.

Consider the example: "The car hit the pole while it was moving"

The interpretations can be

- The car, while moving, hit the pole
- The car the pole while the pole was moving.

The interpretation is preferred than the second because we have model of the world that helps distinguish what logical (or possible) from what is not. To supply to computer model the world is not so easy.

Consider the example: "We saw his duck"

Duck can refer person's bird or to a motion he made.

Semantic ambiguity happens when a sentence contains an ambiguous word or phrase.

4. **Discourse Ambiguity:** Discourse level processing needs shared world or shared knowledge and interpretation is carried out using this context. Anaphoric ambiguity under discourse level.

- **Anaphoric Ambiguity:** Anaphora's are the entities that have been previously introduced into the discourse.

  For example, The horse ran up the hill. It was very steep. It soon got tired.

  The anaphoric reference of 'it' in the two situation cause ambiguity. Steep applies to surface hence 'it' can be hill. Tired applies to animate object hence 'it' can be horse.

5. **Pragmatic Ambiguity:** Pragmatic ambiguity refers to a situation where the context of a phrase gives it multiple meanings. One of the hardest tasks in NLP. The problem involves processing user intention, sentiment, belief world etc. all of which are highly complex tasks.

   Consider the example,

   "Tourist (checking out of the hotel): Waiter, go upstairs to myroom and see it my sandals are there; do not be late; I have to catch the train in 15 minutes."

   Waiter (running upstairs and coming back panting) Yes sir, they are there.

   Clearly, the waiter is falling short of the expectation of the tourist, since he does not understand the pragmatics of the situation.

   Pragmatic ambiguity arises when the statement is not specific, and the context does not provide the information needed to clarify the statement. Information is missing, and must be inferred. Consider the example: "I love you too."

   This can be interpreted as:

   - I love you (as well as liking you)
   - I love you (just like someone else does)
   - I love you (just like you love me)
   - I love you (and I love someone else)

   It is a highly complex task to resolve all these kinds of ambiguities, especially in the upper levels of NLP. The meaning of a word, phrase, or sentence cannot be understood in isolation and contextual knowledge is needed to interpret the meaning, pragmatic and world knowledge is required in higher levels. It is not easy job to create a world model for disambiguation tasks. Linguistic tools and lexical resources are needed for the development of disambiguation techniques. Resourceless languages are lagging behind in these fields compared to resourceful languages in implementation of these techniques. Rule based methods are language specific where as

stochastic or statistical methods are language independent. Automatic resolution of all these ambiguities contains several long standing problems but again development towards full-fledged disambiguation techniques is required which takes care of all the ambiguities. It is very much necessary for the accurate working of NLP applications such as Machine Translation, Information Retrieval, Question Answering etc.

Statistical Approaches of Ambiguity Resolution in Natural Language Processing are:

1. Probabilistic model
2. Part of Speech Tagging

- Rule-Based Approaches
- Mak Model Approaches
- Maxman Entropy Approaches
- HMI Based Taggers

3. Machine Learning Approaches

STAGES IN NLP

There are general five steps in natural language processing

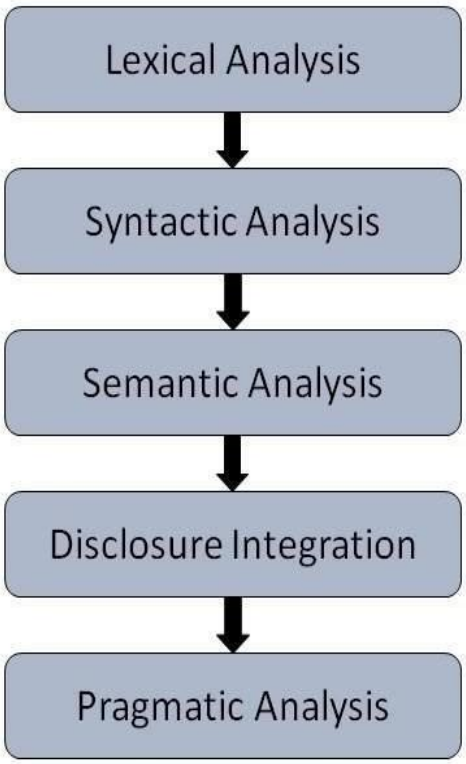


Figure 1.6.1 General five steps of NLP

1) Lexical Analysis:

It is the first stage in NLP. It is also known as morphological analysis. It consists of identifying and analyzing the structure of words. Lexicon of a language means the collection of phrases and words in a language. Lexical analysis is dividing the whole chunk of text into words, sentences, and paragraphs

2) Syntactic Analysis:

Syntactic analysis consists of analysis of words in the sentence for grammar and ordering words in a way that shows the relationship among the words. For example the sentence such as "The school goes to boy" is rejected by English syntactic analyzer.

3) Semantic Analysis:

Semantic analysis is a structure created by the syntactic analyzer which assigns meanings. This component transfers linear sequences of words into structures. It shows how the words are associated with each other. Semantics focuses only on the literal meaning of words, phrases, and sentences. This only draws the dictionary meaning or the real meaning from the given text. The structures assigned by the syntactic analyzer always have assigned meaning

The text is checked for meaningfulness. It is done by mapping syntactic structure and objects in the task domain. E.g. "colorless green idea". This would be rejected by the Symantec analysis as colorless here; green doesn't make any sense.

4) Discourse Integration:

The meaning of any sentence depends upon the meaning of the sentence just before it. Furthermore, it also brings about the meaning of immediately succeeding sentence. For example, "He wanted that", in this sentence the word "that" depends upon the prior discourse context.

5) Pragmatic Analysis:

Pragmatic analysis concerned with the overall communicative and social content and its effect on interpretation. It means abstracting or deriving the meaningful use of language in situations. In this analysis, what was said is reinterpreted on what it truly meant. It contains deriving those aspects of language which necessitate real world knowledge.

E.g., "close the window?" should be interpreted as a request instead of an order.

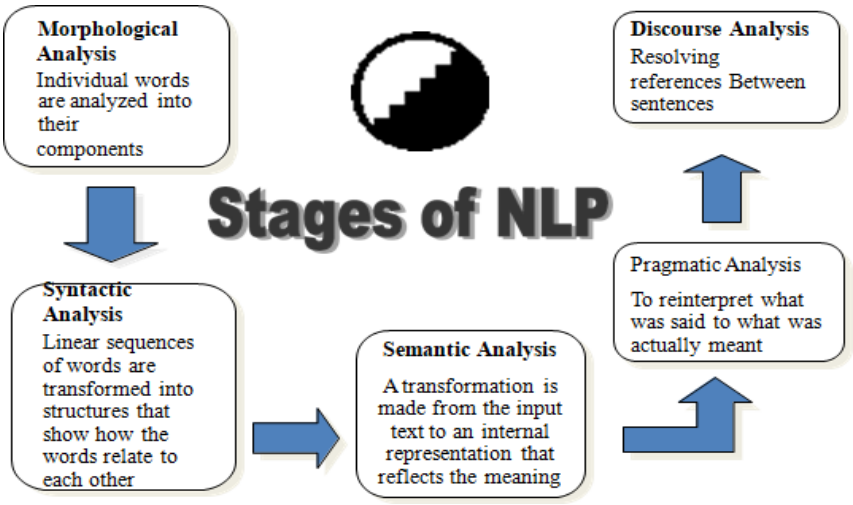


Figure 1.6.2 Stages of NLP

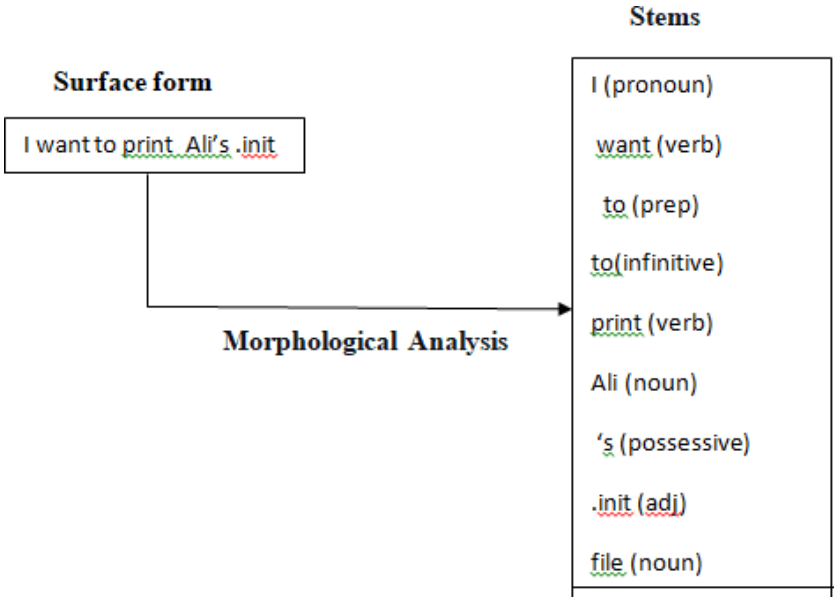
Morphological Analysis:

Consider we have an English interface to an operating system and the following sentence is typed: "I want to print Bill's .init file ".

Morphological analysis must do the following things:

- Pull apart the word "Bill's" into proper noun "Bill" and the possessive suffix "'s".
- Recognize the sequence ".init" as a file extension that is functioning as an adjective in the sentence.

This process will usually assign syntactic categories to all the words in the sentence. Consider the word "prints". This word is either a plural noun or a third person singular verb (he prints).



Syntactic analysis:

This method examines the structure of a sentence and performs detailed analysis of the sentence and semantics of the statement. In order to perform this, the system is expected to have through knowledge of the grammar of the language. The basic unit of any language is sentence, made up of group of words, having their own meanings and linked together to present an idea or thought. Apart from having meanings, words fall under categories called parts of speech. In English languages, there are eight different parts of speech. They are nouns, pronoun, adjectives, verb, adverb, prepositions, conjunction and interjections.

In English language, a sentence S is made up of a noun phrase (NP) and a verb phrase (VP), i.e.

S=NP+VP

The given noun phrase (NP) normally can have an article or delimiter (D) or an adjective (ADJ) and the noun (N), i.e.

NP=D+ADJ+N

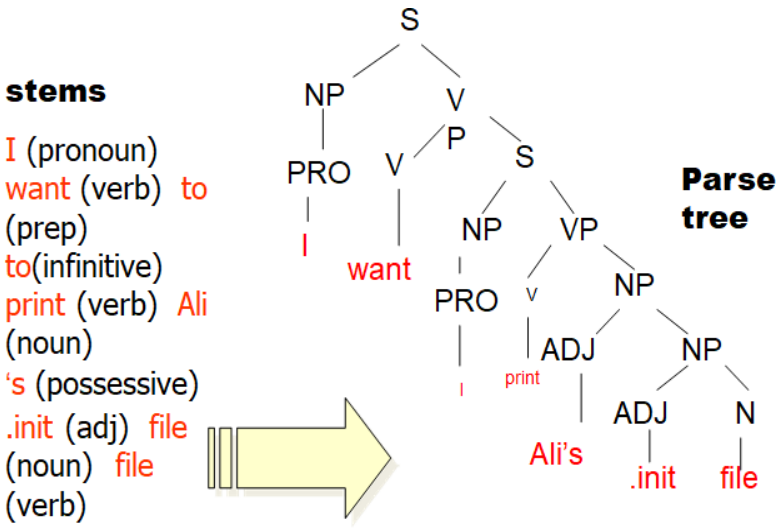
Also a noun phrase may have a prepositional phrase (PP) which has a preposition (P), a delimiter (D) and the noun (N), i.e.

PP=D+P+N

The verb phrase (VP) has a verb (V) and the object of the verb. The object of the verb may be a noun (N) and its determiner, i.e.

VP=V+N+D

These are some of the rules of the English grammar that helps one to construct a small parser for NLP.



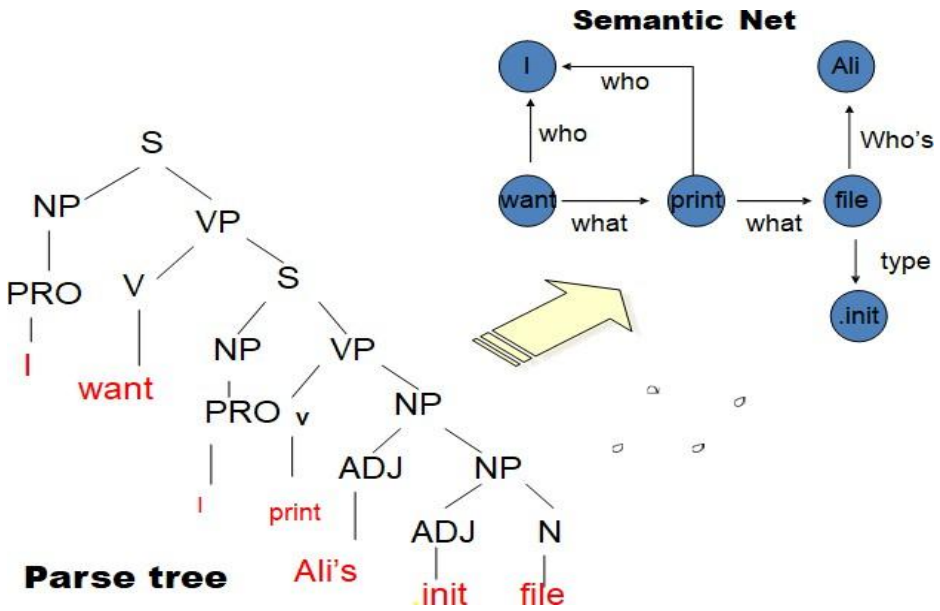
Syntactic analysis must exploit the results of morphological analysis to build a structural description of the sentence. The goal of this process, called parsing, is to convert the flat list of words that forms the sentence into a structure that defines the units that are represented by that flat list. The important thing here is that a flat sentence has been converted into a hierarchical structure and that the structures correspond to meaning units

when semantic analysis is performed. Reference markers are shown in the parenthesis in the parse tree. Each one corresponds to some entity that has been mentioned in the sentence.

Semantic Analysis:

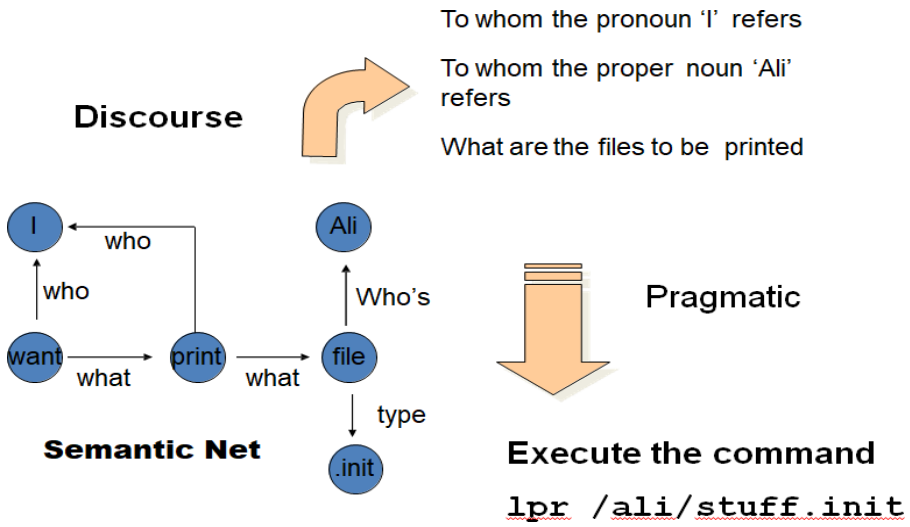
The structures created by the syntactic analyzer assigned meanings. Also, a mapping made between the syntactic structures and objects in the task domain. Moreover, Structures for which no such mapping possible may reject

Semantic analysis must do two important things:
It must map individual words into appropriate objects in the knowledge base or database It must create the correct structures to correspond to the way the meanings of the individual words combine with each other.



Discourse Integration:

The meaning of an individual sentence may depend on the sentences that precede it. And also, may influence the meanings of the sentences that follow it
Specifically we do not know whom the pronoun "I" or the proper noun "Bill" refers to. To pin down these references requires an appeal to a model of the current discourse context, from which we can learn that the current user is USER068 and that the only person named "Bill" about whom we could be talking is USER073.Once the correct referent for Bill is known, we can also determine exactly which file is being referred to.



Pragmatic Analysis

Moreover, The structure representing what said reinterpreted to determine what was actually meant. The final step toward effective understanding is to decide what to do as a results. One possible thing to do is to record what was said as a fact and be done with it. For some sentences, whose intended effect is clearly declarative, that is precisely correct thing to do. But for other sentences, including this one, the intended effect is different. We can discover this intended effect by applying a set of rules that characterize cooperative dialogues. The final step in pragmatic processing is to translate, from the knowledge based representation to a command to be executed by the system.

Results of each of the main processes combine to form a natural language system.

The results of the understanding process are lpr /ali/stuff.init. All of the processes are important in a complete natural language understanding system. Not all programs are written with exactly these components. Sometimes two or more of them collapsed. Doing that usually results in a system that is easier to build for restricted subsets of English but one that is harder to extend to wider coverage

CHALLENGES IN NLP

There are a number of challenges of natural language processing because natural language is ever-evolving and always somewhat ambiguous.

The major challenges around NLP that one must be aware of are as follows:

1. **Language differences.**
   Different languages have not only different sets of vocabulary, but also different types of phrasing, different modes of inflection, and different cultural norms. You can resolve this issue with the help of

"universal" models that can move at least part of what you've learned to other languages. However, you have to spend time updating your NLP system for each additional language.

2. **Training data**

NLP is all about studying language to better understand it. For a person to be proficient, it is essential to spend a substantial amount of time listening, reading, and understanding it, Even the best AI must also spend a significant amount of time reading, listening to, and using a language. The training data given to an NLP system determines its capabilities. If you feed the system bad or questionable data, it's going to learn the wrong things, or learn in an inefficient way.

3. **Development time.**

You also need to think about the development time for an NLP system. The total time taken to develop an NLP system is higher. AI evaluates the data points to process them and use them accordingly. With a shared deep network and many GPUs working together, the training time can be reduced by a few hours. The pre-existing NLP technologies can help in developing the product from scratch.

4. **Phrasing ambiguities.**

Many times it's hard even for another human being to parse out what someone means when they say something ambiguous. There may not be a clear, concise meaning to be found in analysis of their words. In order to resolve this, an NLP system must be able to seek context that can help it understand the phrasing. It may also need to ask the user for clarity.

5. **Misspellings.**

It is common for humans to make spelling mistakes but it can be easily interpreted by human being. But for a machine, misspellings can be harder to detect it, hence it is essential to employ NLP technology to progress and identify the misspellings.

6. **Words with multiple meanings.**

No language is perfect, and most languages have words that can have multiple meanings, depending on the situation. For example, a user who asks, "how are you?" has a totally different aim than a user who asks something like "how do I add a new debit card?" Good NLP tools should be able to discriminate between these utterances with the help of context.

7. **Phrases with multiple intentions.**

Some words and questions actually have multiple meaning, so your NLP system can't oversimplify the situation by interpreting only one of them. For example, a user may say to your chatbot with something like, "I need to cancel my previous order and update my card on file." Your AI needs to be able to distinguish these intentions separately.

8. **False positives and uncertainty.**
   A false positive occurs when an NLP notices a word that should be intelligible or addressable, but cannot be sufficiently answered. The idea is to develop an NLP system that can recognize its own limitations, and use questions to clear up the ambiguity.

9. **Keeping a conversation moving.**
   Many current NLP applications are built on communication between a human and a machine. Accordingly, your NLP AI needs to be able to keep the conversation moving, providing more questions to collect more information or data and always pointing toward a solution.

APPLICATIOS OF NLP

- **Machine Translation**

  Machine translation is basically used to convert text or speech from one natural language to another natural language. Machine translation, an integral part of Natural Language Processing where translation is done from source language to target language preserving the meaning of the sentence. Example: Google Translator

- **Information Retrieval**

  It refers to the human-computer interaction (HCI) that happens when we use a machine to search a body of information for information objects (content) that match our search query. A Person's query is matched against a set of documents to find a subset of 'relevant' document. Examples: Google, Yahoo, Altavista, etc.

- **Text Categorization**

  Text categorization (also known as text classification or topic spotting) is the task of automatically sorting a set of documents into categories (clusters).

Uses of Text Categorization

- Filtering of content
- Spam filtering Identification of document content
- Survey coding

- **Information Extraction** -

Identify specific pieces of information in unstructured or semi-structured textual document. Transform unstructured information in a corpus of documents or web pages into a structured database.

Applied to different types of text:

- Newspaper articles
- Web pages
- Scientific articles
- Newsgroup messages
- Classified ads
- Medical notes

- **Grammar Correction-**

In word processor software like MS-word, NLP techniques are widely used for spelling correction & grammar check.

- **Sentiment Analysis-**
  Sentiment Analysis is also known as opinion mining. It is mainly used on the web to analyse the behaviour, attitude, and emotional state of the sender. This application is implemented through a combination of NLP) and statistics by assigning the values to the text (natural, positive or negative), identify the mood of the context (sad, happy, angry, etc.)

- **Question-Answering systems-**
  Question Answering focuses on constructing systems that automatically answer the questions asked by humans in a natural language. It presents only the requested information instead of searching full documents like search engine. The basic idea behind the QA system is that the users just have to ask the question and the system will retrieve the most appropriate and correct answer for that question.

WORD LEVEL ANALYSIS

REGULAR EXPRESSION

Regular expressions are defined as a sequence of characters that are mainly used to find substrings in a given string. A regular expression (RE) is a language for specifying text search strings. It helps us to match or extract other strings or sets of strings, with the help of a specialized syntax present in a pattern. For Example, extracting all hashtags from a tweet, getting phone numbers, email ID etc from large unstructured text content. Regular Expressions are used in various tasks such as Data pre-processing, Text feature Engineering, Data Extraction, Rule-based information Mining systems, Pattern Matching, Web scraping, etc.

Properties of Regular Expressions

Some of the important properties of Regular Expressions are as follows:

1. Regular Expression (RE) is a formula in a special language, which can be used for specifying a sequence of symbols, simple classes of strings. In simple words, we can say that regular Expression is an algebraic notation for characterizing a set of strings.
2. Regular expression requires two things, one is the pattern that we want to search and the other is a corpus of text or a string from which we need to search the pattern. For example, the Unix command-line tool grep takes a regular expression and returns every line of the input document that matches the expression

Regular Expression can be defined in the following manner:

1. ε is a Regular Expression, which indicates that the language is having an empty string.
2. φ is a Regular Expression which denotes that it is an empty language.
3. If X and Y are Regular Expressions, then the following expressions are also regular.
   - X, Y
   - X.Y(Concatenation of XY)
   - X+Y (Union of X and Y)
   - X*, Y* (Kleen Closure of X and Y)
4. If a string is derived from the above rules then that would also be a regular expression.

How can Regular Expressions be used in NLP?

In NLP, we can use Regular expressions at many places such as,

1. To validate data fields. For example, URLs, dates, email address, abbreviations, etc.
2. To filter a particular text from the whole corpus. For example, disallowed websites, spam etc.
3. To identify particular strings in a text. For example, token boundaries
4. To convert the output of one processing component into the format required for a second component

Basic Regular Expression Patterns

Brackets ([ ]):

They are used to specify a disjunction of characters.

For Examples,

/[A-Z]/ → matches an uppercase letter
/[a-z]/ → matches a lowercase letter
/[0–9]/ → matches a single digit

RE	Match	Example Patterns Matched
/ [A-Z] /	an uppercase letter	"we should call it 'Drenched Blossoms'"
/ [a-z] /	a lowercase letter	"my beans were impatient to be hoed!"
/ [0-9] /	a single digit	"Chapter 1: Down the Rabbit Hole"

For Examples,

/[cC]hirag/ → Chirag or chirag
/[xyz]/ → 'x', 'y', or 'z'
/[1234567890]/ → any digit

Here slashes represent the start and end of a particular expression.

Dash (-):

They are used to specify a range.

RE	Match	Example Patterns
/ [wW] oodchuck/	Woodchuck or woodchuck	"Woodchuck"
/ [abc] /	'a', 'b', *or* 'c'	"In uomini, in soldati"
/ [1234567890] /	any digit	"plenty of 7 to 5"

Caret (^):

They can be used for negation or just to mean ^.

For Examples,

/[^a-z]/ → not an lowercase letter
/[^Cc]/ → neither 'C' nor 'c'
/[^.]/ → not a period
/[c^]/ → either 'c' or '^'
/x^y/ → the pattern 'x^y'

RE	Match (single characters)	Example Patterns Matched
[^A-Z]	not an uppercase letter	"Oyfn pripetchik"
[^Ss]	neither 'S' nor 's'	"I have no exquisite reason for't"
[^\.]	not a period	"our resident Djinn"
[e^]	either 'e' or '^'	"look up ^ now"
a^b	the pattern 'a^b'	"look up a^ b now"

Question mark (?):

It marks the optionality of the previous expression.

For Examples,

/maths?/ → math or maths
/colou?r/ → color or colour

RE	Match	Example Patterns Matched
`woodchucks?`	woodchuck or woodchucks	"woodchuck"
`colou?r`	color or colour	"colour"

Period (.):

Used to specify any character between two expressions.

/beg.n/ → Match any character between which fits between beg and n such as begin, begun

RE	Match	Example Patterns
`/beg.n/`	any character between *beg* and *n*	begin, beg'n, begun

Anchors :

These are special characters that help us to perform string operations either at the beginning or at the end of text input. They are used to assert something about the string or the matching process. The most common anchors are the caret ^ and the dollar sign $.

i) Caret character '^'

It specifies the start of the string. For a string to match the pattern, the character followed by the '^' in the pattern should be the first character of the string.

For Examples,
^The: Matches a string that starts with 'The'

ii) Dollar character '$'

It specifies the end of the string. For a string to match the pattern, the character that precedes the '$' in the pattern should be the last character in the string.

For Examples,
end$: Matches a string that ends with 'end'
^The end$': Exact string match (starts and ends with 'The end')
roar: Matches a string that has the text roar in it.

Quantifiers:

They allow us to mention and control over how many times a specific character(s) pattern should occur in the given text.
The most common Quantifiers are: *, +, ? and { }

RE	Match
*	zero or more occurrences of the previous char or expression
+	one or more occurrences of the previous char or expression
?	exactly zero or one occurrence of the previous char or expression
{n}	*n* occurrences of the previous char or expression
{n,m}	from *n* to *m* occurrences of the previous char or expression
{n,}	at least *n* occurrences of the previous char or expression

For Examples,

abc*: matches a string that has 'ab' followed by zero or more 'c'.
abc+: matches 'ab' followed by one or more 'c'
abc?: matches 'ab' followed by zero or one 'c'
abc{2}: matches 'ab' followed by 2 'c'
abc{2, }: matches 'ab' followed by 2 or more 'c'
abc{2, 5}: matches 'ab' followed by 2 upto 5 'c'
a(bc)*: matches 'a' followed by zero or more copies of the sequence 'bc'

FINITE AUTOMATA

The term automata, derived from the Greek word "αὐτόματα. Automaton may be defined as an abstract self-propelled computing device that follows a predetermined sequence of operations automatically. An automaton having a finite number of states is called a Finite Automaton (FA) or Finite State automata (FSA).

The finite-state automaton is not only the mathematical device used to implement regular expressions, but also one of the most significant tools of computational linguistics. Variations of automata such as finite- state transducers, Hidden Markov Models, and *N*-gram grammars are important components of the speech recognition and synthesis, spell-checking, and information-extraction applications.

Mathematically, an automaton can be represented by a 5-tuple (Q, Σ, δ, q0, F),

where −

- Q is a finite set of states.
- Σ is a finite set of symbols, called the alphabet of the automaton.
- δ is the transition function
- q0 is the initial state from where any input is processed (q0 ∈ Q).
- F is a set of final state/states of Q (F ⊆ Q).

An RE is one way of describing finite state automata (FSA). An RE is one way of characterizing a particular kind of formal language called a regular language. Regular grammar is the equivalent ways of describing regular languages.

Relation between Finite Automata, Regular Grammars and Regular Expressions

- As we know that finite state automata are the theoretical foundation of computational work and regular expressions is one way of describing them.
- We can say that any regular expression can be implemented as FSA and any FSA can be described with a regular expression.
- On the other hand, regular expression is a way to characterize a kind of language called regular language. Hence, we can say that regular language can be described with the help of both FSA and regular expression.
- Regular grammar, a formal grammar that can be right-regular or left-regular, is another way to characterize regular language.

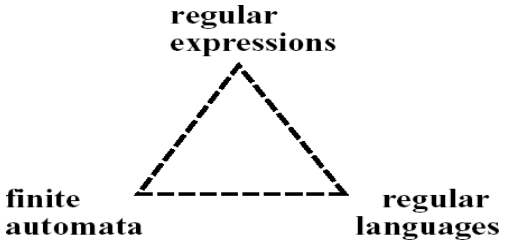


Figure 3.2.1 Relation between Finite Automata, Regular languages and Regular Expressions

Finite state automation is of two types:

- Deterministic Finite automation (DFA)
- Non-deterministic Finite Automation (NDFA)

1. Deterministic Finite automation (DFA)

It may be defined as the type of finite automation wherein, for every input symbol we can determine the state to which the machine will move. It has a finite number of states that is why the machine is called Deterministic Finite Automaton (DFA).

Mathematically, a DFA can be represented by a 5-tuple (Q, Σ, δ, q0, F), where −

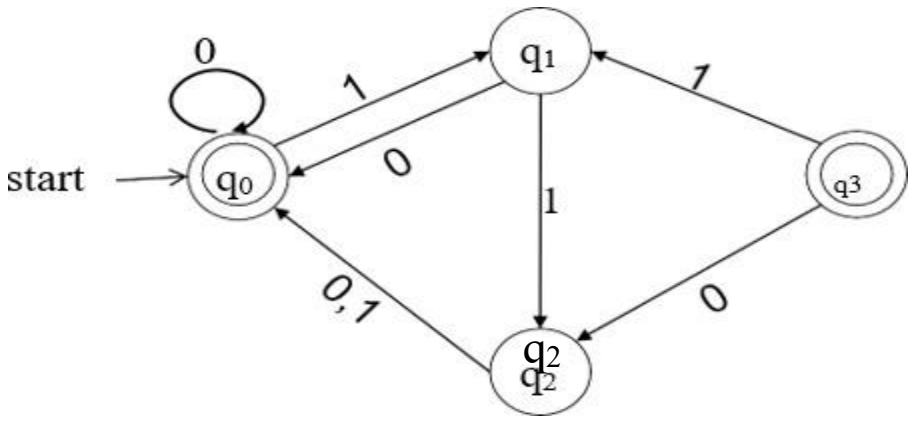
- Q is a finite set of states.
- Σ is a finite set of symbols, called the alphabet of the automaton.
- δ is the transition function where $\delta: Q \times \Sigma \rightarrow Q$.
- q0 is the initial state from where any input is processed ($q0 \in Q$).
- F is a set of final state/states of Q ($F \subseteq Q$).

Whereas graphically, a DFA can be represented by diagraphs called state diagrams where −

The states are represented by vertices.
The transitions are shown by labeled arcs.
The initial state is represented by an empty incoming arc.
The final state is represented by a double circle.

Construct the state diagram for the finite-state automata M= (Q, Σ, δ, q0, F),, where Q = {q_0, q_1, q_2, q_3}, Σ = {0,1}, F= {q_0, q_3} and the transition function δ is given in the following table.

State	0	1
q0	q_0	q_1
q1	q_0	q_2
q2	q_0	q_0
q3	q_2	q_1



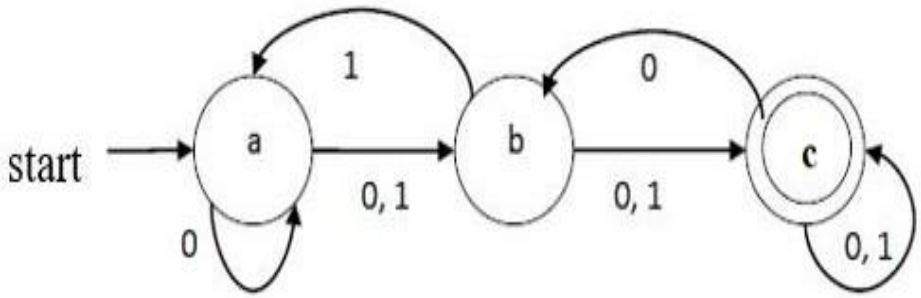
2. Non-deterministic Finite Automation (NDFA)

It may be defined as the type of finite automation where for every input symbol we cannot determine the state to which the machine will move i.e. the machine can move to any combination of the states. It has a finite number of states that is why the machine is called Non-deterministic Finite Automation (NDFA).

Mathematically, NDFA can be represented by a 5-tuple (Q, Σ, δ, q0, F), where −

- Q is a finite set of states.
- Σ is a finite set of symbols, called the alphabet of the automaton.
  δ :-is the transition function where δ: $Q \times \Sigma \rightarrow 2^Q$.
- q0 :-is the initial state from where any input is processed (q0 ∈ Q).
- F :-is a set of final state/states of Q (F ⊆ Q).

The finite state automaton M= (Q, Σ, q0, δ, F), where Q= {a, b, c }, Σ ={0,1}, F={c}, and δ is depicted graphically as follows:



State	0	1
a	a, b	b
b	c	a, c
c	b, c	c

FINITE STATE MORPHOLOGICAL PARSING

To know the structure about a word when we perform the morphological parsing for that word. Given a surface form (input form), e.g., "going" we might produce the parsed form: verb-go + gerund-ing. Morphological parsing can be done with the help of finite-state transducer. A morpheme is a meaning bearing unit of any language. For example, cats: has two morphemes, cat, -s.

The objective of the morphological parsing is to produce output lexicons for single input lexicon. Our aim is to take input forms like in the first column of Table 3.3.1 and produce the output like in second column. The second column contains the stem of each word as well as mixed morphological features. These features include, like N (noun), V (verb), specify additional information about the word stem, e.g., +N means that word is noun, +SG means singular, +PL for plural, etc.

Table 3.3.1 Mapping of input word to Morphemes.

Input Words	Morphological parsed output
cats	cat *+N +PL*
cat	cat *+N +SG*
cities	city *+N +PL*
geese	goose *+N +PL*
goose	goose *+V +3SG*
caught	catch *+V +PAST-Part*

To build morphological parser, we need at least following database:

1. Lexicon: List of stems, and affixes, plus additional information about them, like +N, +V.
2. Morphotactics rules: Rules about ordering of morphemes in a word, e.g. -ed is followed after a verb (e.g., worked, studied), un (undo) precede a verb, for example, unlock, untie, etc.
3. Orthographic rules (spelling): For combining morphemes, e.g., city+ -s gives cities and not citys.

BUILDING A FINITE-STATE LEXICON

A lexicon is a repository for words. They are grouped according to their main categories like noun, adjective, verb, adverb etc. They may be divided in to sub categories like regular-nouns, irregular-singular nouns, irregular-plural nouns. The simplest possible lexicon would consist of an explicit list of every word of the language (every word, i.e., including abbreviations ("AAA") and proper names ("Joy" or "Bebo")) as follows:

a, AAA, AA, Aachen, aardvark, aardwolf, aba, abaca, aback,

The simplest way to create a morphological parser put all possible words in to lexicon. As it's often impossible, for the various reasons, to list every word in the language, computational lexicons are usually structured with a list of each of the stems and affixes of the language together with a representation of the morphotactics that tells us how they can fit together. There are many ways to model morphotactics; one of the most common is the finite state automaton. A very simple finite-state model for English nominal inflection might look like Fig. 3.4.1.

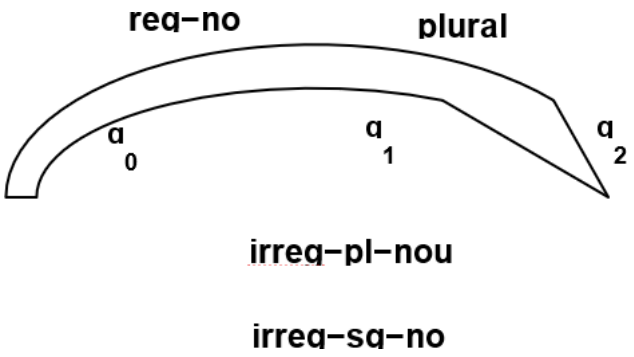


Figure 3.4.1 A finite-state automaton for English nominal inflection.

The FSA in Fig. 3.4.1 assumes that the lexicon includes regular nouns (reg-noun) that take the regular -s plural (e.g. dog, fox, cat, and aardvark). The lexicon also includes irregular noun forms that don't take -s, both singular irreg-sg-noun (goose, mouse) and plural irreg-pl-noun (geese, mice).

reg-noun	irreg-pl-noun	irreg-sg-noun	plural
fox cat aardvark	geese sheep mice	goose sheep mouse	-s

A similar model for English verbal inflection might look like Fig. 3.4.2

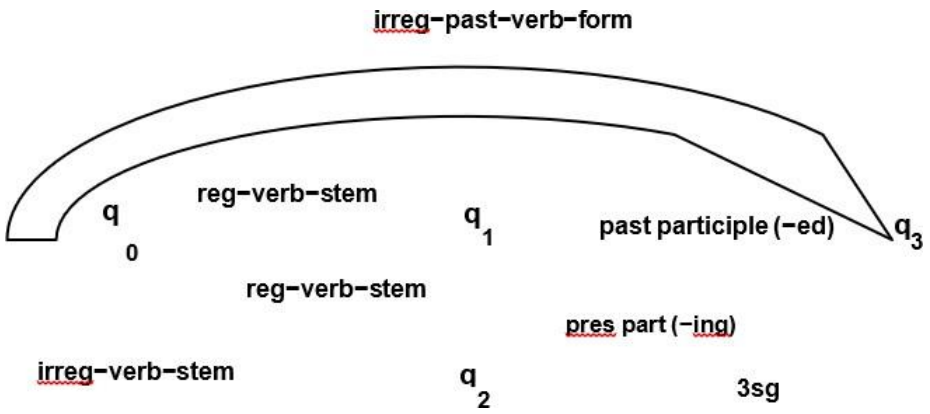


Figure 3.4.2 A finite-state automaton for English verbal inflection

This lexicon has three stem classes (reg-verb-stem, irreg-verb-stem, and irreg-past- verb-form), plus four more affix classes (-ed past, -ed participle, -ing participle, and third singular -s):

reg-verb-stem	irreg-verb-stem	irreg-past-verb	past	past-part	pres-part	3sg
walk fry talk impeach	cut speak sing	caught ate eaten sang	-ed	-ed	-ing	-s

English derivational morphology is significantly more complex than English inflectional morphology, and so automata for modeling English derivation tend to be quite complex. An initial hypothesis might be that adjectives can have an optional prefix (un-), an obligatory root (big, cool, etc.) and an optional suffix (-er, -est, or -ly). This might suggest the FSA in Fig. 3.4.3.

big, bigger, biggest, cool, cooler, coolest, coolly
happy, happier, happiest, happily red, redder, reddest
unhappy, unhappier, unhappiest, unhappily real, unreal, really
clear, clearer, clearest, clearly, unclear, unclearly

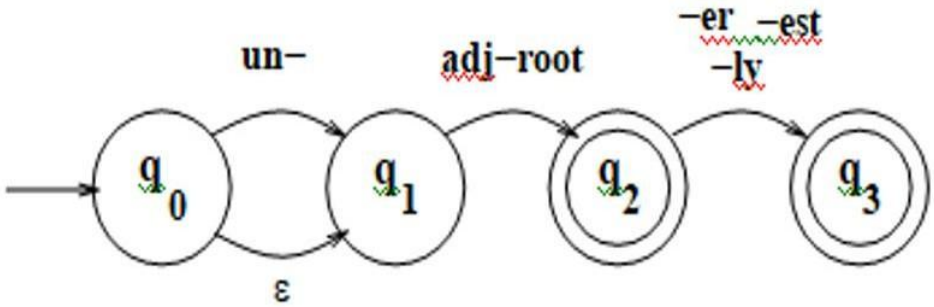


Figure 3.4.3 An FSA for a fragment of English adjective morphology

An FSA for another fragment of English adjective morphology is shown in Fig. 3.4.4.

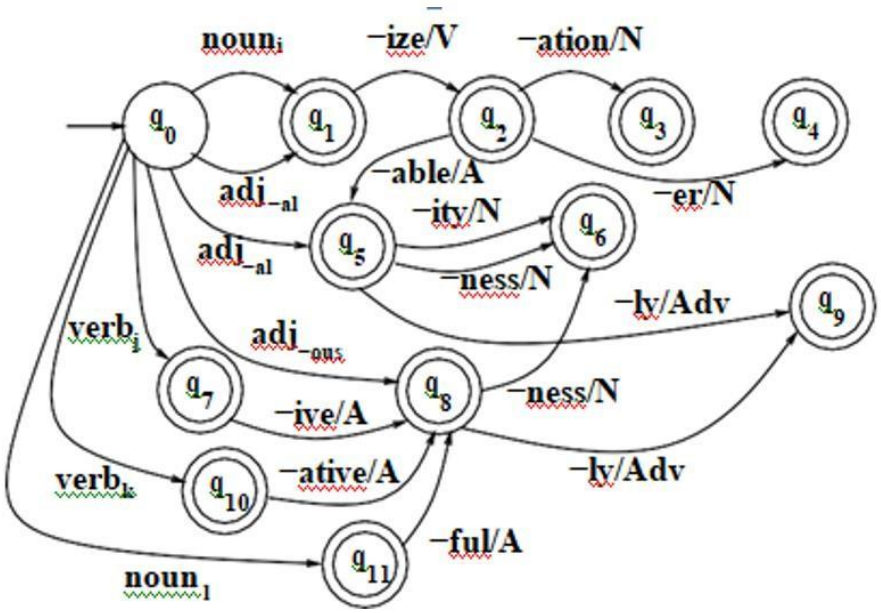


Figure 3.4.4 An FSA for another fragment of English derivational morphology.

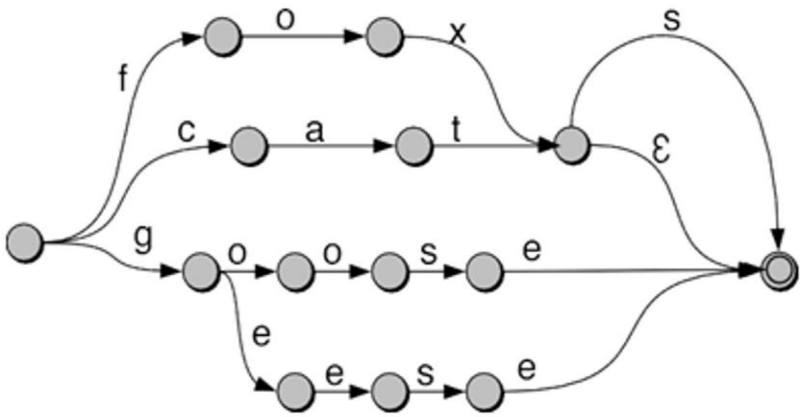


Figure 3.4.5 Expanded FSA for a few English nouns with their inflection

Expanded FSA for a few English nouns with their inflection is shown in Fig, 3.4.5. Note that this automaton will incorrectly accept the input foxs.

FINITE STATE TRANSDUCERS (FST)

FSAs can be used to recognise particular patterns but don't, by themselves, allow for any analysis of word forms. Hence for morphology, we use finite state transducers (FSTs) which allow the surface structure to be mapped into the list of morphemes. FSTs are useful for both analysis and generation, because the mapping is bidirectional. This approach is known as two-level morphology.

FST is a type of finite automaton which maps between two sets of symbols. We can visualize an FST as a two-tape automaton which recognizes or generates pairs of strings. We can do this by labelling each arc in the finite-state machine with two symbol strings, one from each tape. Fig 3.5.1 shows an example of an FST where each arc is labelled by an input and output string, separated by a colon.

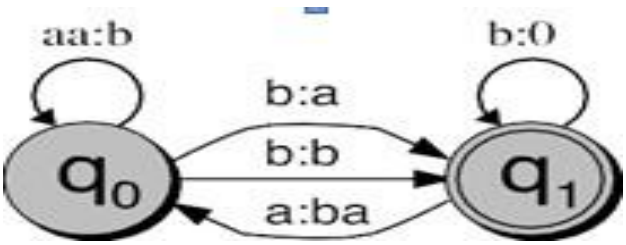


Figure 3.5.1 A Finite-state transducer

FSA defines a formal language by defining a set of strings, an FST defines a relation between sets of strings. In short, FST is as a machine that reads one string and generates another.

Four-fold way of thinking about transducer:

i. **FST as recognizer:** a transducer that takes a pair of strings as input and outputs accept if the string-pair is in the string-pair language, and reject if it is not.

ii. **FST as generator:** a machine that outputs pairs of strings of the language. Means the output is a yes or no, and a pair of output strings.

iii. **FST as translator:** a machine that reads a string and outputs another string

iv. **FST as set relater:** a machine that computes relations between sets.

For morphological parsing (and for many other NLP applications), we can apply the FST as translator metaphor, taking as input a string of letters and producing as output a string of morphemes.

Formal definition an FST can be defined with 7 parameters:

Q: a finite set of N states q0, q1, . . . , qN−1

Σ: a finite set corresponding to the input alphabet

Δ: a finite set corresponding to the output alphabet

$q_0 \in Q$: start state

$F \subseteq Q$: set of final states

δ(q, w): the transition function between states; Given a state $q \in Q$ and a string $w \in \Sigma^*$,

δ(q, w) returns a set of new states $Q' \in Q$. δ is thus a function from $Q \times \Sigma*$ to 2^Q (because there are 2^Q possible subsets of Q). δ returns a set of states rather than a single state because a given input may be ambiguous in which state it maps to.

σ(q, w): the output function giving the set of possible output strings for each state and

input. Given a state $q \in Q$ and a string $w \in \Sigma^*$, σ(q, w) gives a set of output

strings, each a string $o \in \Delta*$. σ is thus a function from $Q \times \Sigma*$ to $2^{\Delta}*$

Where FSAs are isomorphic to regular languages, FSTs are isomorphic to regular relations. Regular relations are sets of pairs of strings, a natural extension of the regular languages, which are sets of strings. Like FSAs and regular languages, FSTs and regular relations are closed under union, although in general they are not closed under difference, complementation and.

Besides union, FSTs have two additional closure properties that turn out to be extremely useful:

- **Inversion:** The inversion of a transducer T (T^{-1}) simply switches the input and output labels. Thus if T maps from the input alphabet I to the output alphabet O, T^{-1} maps from O to I.

  Inversion is useful because it makes it easy to convert a FST-as-parser into an FST- as-generator.

  T = {(a, a1), (a, a2), (b, b1), (c, c1), (c, a)}

  T^{-1} = {(a1, a), (a2, a), (b1, b), (c1, c), (a, c)

- **Composition**: If T1 is a transducer from I1 to O1 and T2 a transducer from O1 to O2, then T1 ∘ T2 maps from I1 to O2.

Composition is useful because it allows us to take two transducers that run in series and replace them with one more complex transducer. Composition works as in algebra; applying T1 ∘ T2 to an input sequence S is identical to applying T1 to S and then T2 to the result; thus T1 ∘ T2(S) = T2 (T1(S)).

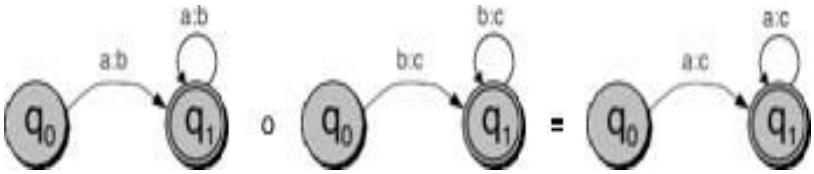


Fig. 3.5.2, for example, shows the composition of [a:b]+ with [b:c]+ to produce [a:c]+.

Figure 3.5.2 The composition of [a: b]+ with [b:c]+ to produce [a:c]+.

The projection of an FST is the FSA that is produced by extracting only one side of the relation. We can refer to the projection to the left or upper side of the relation as the upper or first projection and the projection to the lower or right side of the relation as the lower or second projection

Part-Of-Speech Tagging

Tagging is a kind of classification that may be defined as the automatic assignment of description to the tokens. Here the descriptor is called tag, which may represent one of the part-of-speech, semantic information and so on.

Now, if we talk about Part-of-Speech (PoS) tagging, then it may be defined as the process of assigning one of the parts of speech to the given word. It is generally called POS tagging. In simple words, we can say that POS tagging is a task of labelling each word in a sentence with its appropriate part of speech. We already know that parts of speech include nouns, verb, adverbs, adjectives, pronouns, conjunction and their sub-categories.

Most of the POS tagging falls under Rule Base POS tagging, Stochastic POS tagging and Transformation based tagging.

5.0 What are the parts of Speech?

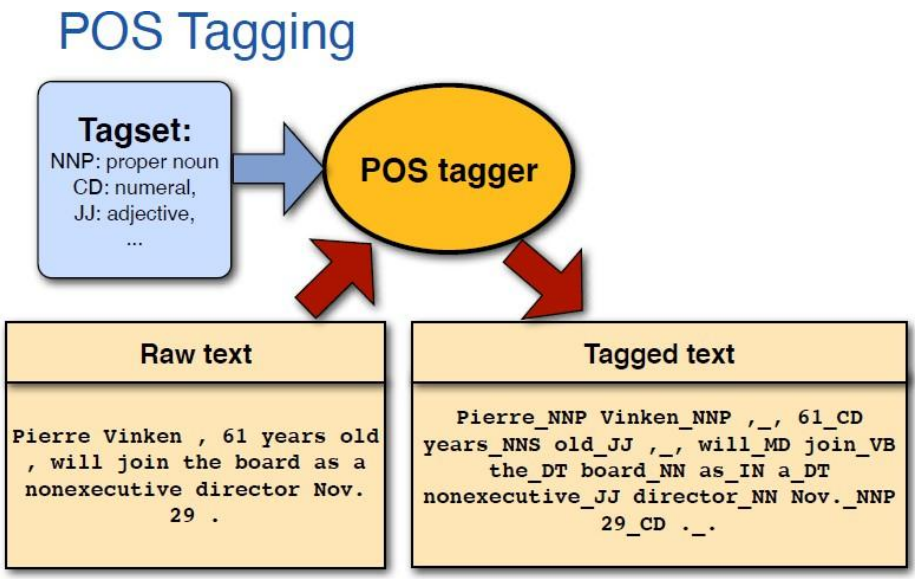
POS tagging words often have more than one POS: - The back door (adjective) - On my back (noun) - Winning back the voters (particle) - Promised to support the bill (verb) The POS tagging task: determine the POS tag for all tokens in a sentence. Due to ambiguity (and unknown words), we cannot rely on a dictionary to look up the correct POS tags.

Why POS tagging?

POS Tagging is one of the first steps in the NLP pipeline (right after tokenization, segmentation). POS tagging is traditionally considered a prerequisite for further analysis:

Syntactic parsing: what words are in the sentence?

Information extraction: finding names, dates, relationships, etc. NB: Although many neural models do not use POS tagging, it is still important to understand what makes POS tagging difficult (or easy) and how the basic models and algorithms work.



Creating a POS Tagger :To deal with ambiguity and coverage, POS taggers rely on learned models. For a new language (or domain) Step 0: Define a POS tag set Step 1: Annotate a corpus with these tags For a well-researched language (and domain):

Step 1: Obtain a POS -tagged corpus For any language....:

Step 2: Choose a POS tagging model (e.g., an HMM) Step 3: Train your model on your training corpus Step 4: Evaluate your model on your test corpus

Define a tag set We need to define an inventory of labels for the word classes (i.e., the tag set) - Most taggers are based on models that need to be trained on annotated (tagged) corpora.

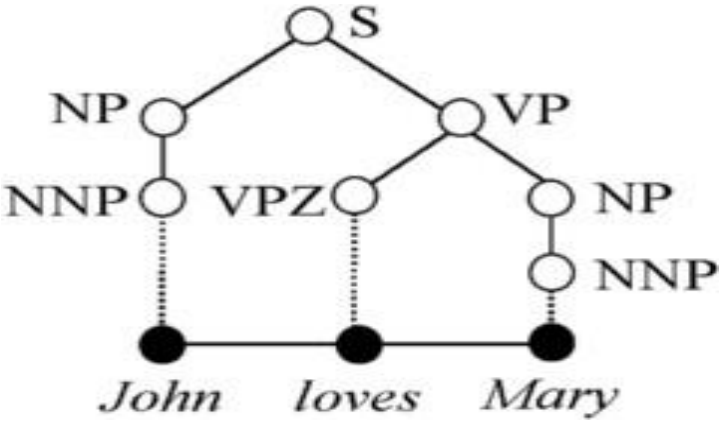
- Evaluation also requires annotated corpora. - Since annotation by humans is expensive and time-consuming, tag sets used in a few existing corpora are becoming the de facto standard. - Tag sets must capture semantically or syntactically important distinctions that can be easily made by trained human annotators.

5.1 Tag set for English (Penn Treebank)

In linguistics, a Treebank is a parsed text corpus that annotates syntactic or semantic sentence structure. The construction of parsed corpora in the early 1990s revolutionized computational linguistics, which benefitted from large-scale empirical data.

What is Treebank NLP?

A Treebank is **a collection of syntactically annotated sentences in which the annotation has been manually checked** so that the Treebank can serve as a training corpus for natural language parsers, as a repository for linguistic research, or as an evaluation corpus for NLP systems.



Tag	Description
ADJ	Adjective
ADV	Adposition
ADP	Adverb
AUX	Auxiliary
CCONJ	Coordinating Conjunction
DET	Determiner
INTJ	Interjection
NOUN	Noun
NUM	Numeral
PART	Particle
PRON	Pronoun
PROPN	Proper Noun
PUNCT	Punctuation
SCONJ	Subordinating Conjunction
SYM	Symbol
VERB	Verb
X	Other

5.3 Rule-based POS Tagging

One of the oldest techniques of tagging is rule-based POS tagging. Rule-based taggers use dictionary or lexicon for getting possible tags for tagging each word. If the word has more than one possible tag, then rule-based taggers use hand-written rules to identify the correct tag. Disambiguation can also be performed in rule-based tagging by analyzing the linguistic features of a word along with its preceding as well as following words. For example, suppose if the preceding word of a word is article then word must be a noun.

As the name suggests, all such kind of information in rule-based POS tagging is coded in the form of rules. These rules may be either −

- Context-pattern rules
- Or, as Regular expression compiled into finite-state automata, intersected with lexically ambiguous sentence representation.

We can also understand Rule-based POS tagging by its two-stage architecture −

- First stage − In the first stage, it uses a dictionary to assign each word a list of potential parts-of-speech.
- Second stage − In the second stage, it uses large lists of hand-written disambiguation rules to sort down the list to a single part-of-speech for each word.

Properties of Rule-Based POS Tagging

Rule-based POS taggers possess the following properties −

- These taggers are knowledge-driven taggers.
- The rules in Rule-based POS tagging are built manually.
- The information is coded in the form of rules.
- We have some limited number of rules approximately around 1000.
- Smoothing and language modeling is defined explicitly in rule-based taggers.

5.3.1 Stochastic POS Tagging

Another technique of tagging is Stochastic POS Tagging. Now, the question that arises here is which model can be stochastic. The model that includes frequency or probability (statistics) can be called stochastic. Any number of different approaches to the problem of part-of-speech tagging can be referred to as stochastic tagger.

The simplest stochastic tagger applies the following approaches for POS tagging −

Word Frequency Approach

In this approach, the stochastic taggers disambiguate the words based on the probability that a word occurs with a particular tag. We can also say that the tag encountered most frequently with the word in the training set is the one assigned to an ambiguous instance of that word. The main issue with this approach is that it may yield inadmissible sequence of tags.

Tag Sequence Probabilities

It is another approach of stochastic tagging, where the tagger calculates the probability of a given sequence of tags occurring. It is also called n-gram approach. It is called so because the best tag for a given word is determined by the probability at which it occurs with the n previous tags.

Properties of Stochastic POST Tagging

Stochastic POS taggers possess the following properties −

- This POS tagging is based on the probability of tag occurring.
- It requires training corpus
- There would be no probability for the words that do not exist in the corpus.
- It uses different testing corpus (other than training corpus).
- It is the simplest POS tagging because it chooses most frequent tags associated with a word in training corpus.

Transformation-based Tagging

Transformation based tagging is also called Brill tagging. It is an instance of the transformation-based learning (TBL), which is a rule-based algorithm for automatic tagging of POS to the given text. TBL, allows us to have linguistic knowledge in a readable form, transforms one state to another state by using transformation rules.

It draws the inspiration from both the previous explained taggers − rule-based and stochastic. If we see similarity between rule-based and transformation tagger, then like rule-based, it is also based on the rules that specify what tags need to be assigned to what words. On the other hand, if we see similarity between stochastic and transformation tagger then like stochastic, it is machine learning technique in which rules are automatically induced from data.

Working of Transformation Based Learning(TBL)

In order to understand the working and concept of transformation-based taggers, we need to understand the working of transformation-based learning. Consider the following steps to understand the working of TBL

- **Start with the solution** − The TBL usually starts with some solution to the problem and works in cycles.

5.3 Rule-based POS Tagging

choose the most beneficial transformation.

- **Apply to the problem** − The transformation chosen in the last step will be applied to the problem.

The algorithm will stop when the selected transformation in step 2 will not add either more value or there are no more transformations to be selected. Such kind of learning is best suited in classification tasks.

Advantages of Transformation-based Learning (TBL)

The advantages of TBL are as follows −

- We learn small set of simple rules and these rules are enough for tagging.
- Development as well as debugging is very easy in TBL because the learned rules are easy to understand.
- Complexity in tagging is reduced because in TBL there is interlacing of machinelearned and human-generated rules.
- Transformation-based tagger is much faster than Markov-model tagger.

Disadvantages of Transformation-based Learning (TBL)

The disadvantages of TBL are as follows −

- Transformation-based learning (TBL) does not provide tag probabilities.
- In TBL, the training time is very long especially on large corpora.

Issues-Multiple tags & words

The main problem with POS tagging is ambiguity. In English, many common words have multiple meanings and therefore multiple POS . The job of a POS tagger is to resolve this ambiguity accurately based on the context of use. For example, the word "shot" can be a noun or a verb